

Delta Engine

Cloud-Native Trade Copier SaaS

Full System Architecture and Product Requirement Document (PRD)

Field	Value
Product Type	Managed trade execution, copying, risk-control, and analytics infrastructure platform
Primary Market	Forex, CFD, prop-firm, signal-provider, and multi-account traders
Initial Platform	MetaTrader 5 (MT5)
Future Platforms	MT4, cTrader, DXTrade, MatchTrader, TradeLocker, TradingView webhooks, NinjaTrader
Version	1.0 Full Product PRD
Status	Ready for Phase 1 technical experimentation
Date	May 2026

Core thesis: Delta Engine is not mainly a dashboard app. It is a managed MetaTrader worker infrastructure product with a SaaS control layer on top.

1. Executive Summary

Delta Engine is a cloud-native trade copier SaaS that allows traders to connect one or more master trading accounts and replicate trades into multiple follower accounts with configurable risk, lot sizing, symbol mapping, execution rules, and audit tracking.

The platform's long-term goal is to become a professional-grade copying infrastructure layer for retail traders, prop-firm traders, account managers, signal providers, and trading communities.

Unlike traditional local trade copiers that require users to install Expert Advisors, rent VPS machines, or keep personal trading terminals online, Delta Engine will provide a managed cloud environment where the system handles terminal uptime, connection monitoring, execution routing, risk controls, and account-level protection.

The first build will focus on MT5-to-MT5 cloud-hosted copying using Python-controlled MetaTrader 5 terminal workers.

2. Product Vision

2.1 Vision Statement

Delta Engine will be the control center for serious multi-account traders. A trader should be able to log in, connect accounts, choose one account as the master, define follower rules, set risk limits, and let Delta Engine handle copying with transparency, safety, and speed.

2.2 Product Philosophy

- **Reliability before speed:** A fast copier that misfires is useless. The first goal is accurate copying, clean state tracking, and safe execution.
- **Risk-first architecture:** The system must prevent bad trades, over-sizing, over-exposure, and prop-firm rule violations before they happen.
- **Cloud convenience:** Users should not need to manage their own VPS or keep a laptop running.
- **Audit everything:** Every copied trade, failed trade, skipped trade, SL/TP modification, risk lockout, and broker error should be logged.
- **Build narrow first, expand later:** MT5 first. Then MT4 and other platforms after the core engine is proven.

3. Market Positioning

3.1 Competitor Lessons

Platform	Main Strength	Main Weakness
Tradecopia	Low-latency, local-first, strong preemptive risk controls, fixed fee economics	Smaller public review footprint, requires local setup for d
TradersConnect	Cloud-native, zero-install, broad platform coverage, strong public review, broad	credential custody, MT4/MT5 discontinuation trust

Delta Engine should position itself as a cloud-native trade copier with serious risk controls, transparent execution logs, and trader-friendly pricing.

3.2 Target Users

- **Forex prop-firm traders:** running multiple evaluation or funded accounts; need consistent replication and drawdown protection.
- **Retail multi-account traders:** copying one strategy across multiple personal accounts with different risk levels.
- **Signal providers:** future role for users who want to send trades to subscribers.
- **Account managers:** users who need controlled multi-account execution, audit logs, and performance reporting.

4. Product Scope

4.1 MVP Scope

Area	MVP Decision
MVP Name	Delta Engine Core
Platform	MT5 only
Copy Direction	MT5 master to MT5 follower
Deployment	Local developer machine first, then controlled cloud/VPS environment
Trading Environment	Demo accounts first

MVP Includes

- User dashboard
- Add MT5 account
- Connect MT5 account through worker
- Select master account
- Select follower account
- Create copier relation
- Copy market buy/sell orders
- Copy SL and TP
- Copy manual close
- Fixed lot mode
- Multiplier mode
- Symbol mapping
- Basic account health monitoring
- Basic daily loss lockout
- Execution logs
- Error logs

MVP Excludes

- MT4

- cTrader
- DXTrade
- MatchTrader
- TradeLocker
- TradingView alerts
- AI journal
- Signal marketplace
- Subscription billing
- Native mobile app
- Public API
- HFT claims
- Guaranteed sub-100ms latency

4.2 Full Future Product Scope

Module	Description
Copier Engine	Core master-to-follower trade replication
Risk Engine	Daily loss, max loss, equity stop, drawdown, trade frequency, symbol restrictions
Symbol Mapper	Maps symbols across brokers, suffixes, prefixes, and contract specs
Account Manager	Adds, authenticates, monitors, and reconnects trading accounts
Terminal Worker Fabric	Runs managed trading terminal instances
Analytics Engine	P&L, win rate, drawdown, latency, slippage, account comparison
AI Journal	Summarizes trades, mistakes, behavior, and risk patterns
Billing Engine	Subscription plans, account limits, and Stripe billing
Notification Engine	Email, Telegram, Discord, SMS, push, and webhook notifications
Public API	API keys, webhooks, sandbox, and signal routing
Signal Marketplace	Signal providers can publish strategies to followers
Admin Console	Internal support, user inspection, account health, worker status
Security Layer	Audit logs, data deletion, credential encryption, and access controls

5. System Architecture

5.1 High-Level Architecture

User Browser

- > Next.js Web Dashboard
- > FastAPI Control API
- > Supabase Auth + PostgreSQL
- > Worker Orchestrator
- > Managed MT5 Terminal Workers
- > Broker Servers

5.2 Internal Execution Flow

Master MT5 Account

- > Master Worker detects trade event
- > Signal Normalizer
- > Copier Relation Lookup
- > Risk Engine
- > Lot Sizing Engine
- > Symbol Mapper
- > Follower Execution Worker
- > Broker Execution
- > Execution Logger
- > Dashboard Update

5.3 Core Components

Next.js Dashboard

The dashboard is the user control surface. Users can sign up, log in, connect trading accounts, view account status, create copier relations, configure lot sizing, configure risk rules, start/stop copying, view copied trades, inspect errors, and view account performance.

Supabase Layer

Supabase handles authentication, user profiles, PostgreSQL database, Row-Level Security, realtime updates, storage for non-sensitive files, and admin-side data access.

FastAPI Control API

FastAPI handles JWT validation, account creation, credential encryption, copier relation management, worker commands, risk configuration, execution log APIs, admin APIs, and future public APIs.

Worker Orchestrator

The orchestrator assigns account sessions to available workers. It starts and stops terminal workers, monitors health, detects stale sessions, schedules reconnects, and manages per-user/account isolation.

Managed MT5 Terminal Workers

Each worker controls one or more MetaTrader 5 terminal sessions. Workers initialize MT5, authenticate trading accounts, fetch account info, fetch positions/orders, detect trade changes, send follower orders, update SL/TP, close follower positions, return broker error codes, and report health status.

6. Core Technical Strategy

6.1 Start with MT5

MT5 is the best first platform because Python-side experimentation is more realistic than MT4. The first technical milestone is not the dashboard. The first milestone is to connect Python to MT5, read account state, read open positions, place a demo trade, and close a demo trade.

6.2 Worker-First Development

- Python MT5 connection script
- Account info reader
- Position reader
- Order sender

- Position closer
- State-diff trade detector
- Master/follower copier script
- Risk checks
- Database persistence
- API layer
- Dashboard

7. User Roles

Role	Permissions
Trader User	Create account, connect trading accounts, configure copier, view logs, configure risk, start/stop copying
Signal Provider	Future role: create signal channel, connect master account, approve followers, view subscriber metrics
Follower Subscriber	Future role: subscribe to a signal, connect follower account, set personal risk limits, disconnect anytime
Admin	View users, worker health, failed jobs, non-sensitive logs, disable accounts, suspend users, trigger worker restart

Admins must not be able to view raw broker passwords.

8. Functional Requirements

8.1 Authentication

- FR-AUTH-1: Users must be able to register with email and password.
- FR-AUTH-2: Users must be able to log in and log out.
- FR-AUTH-3: Users must be able to reset password.
- FR-AUTH-4: Frontend requests to FastAPI must include a Supabase JWT.
- FR-AUTH-5: FastAPI must validate JWTs before allowing access to protected routes.

8.2 Trading Account Management

- FR-ACC-1: Users must be able to add a trading account with platform type, login, broker server, password, label, and account role preference.
- FR-ACC-2: The system must encrypt broker passwords before storing them.
- FR-ACC-3: The frontend must never receive a stored password after account creation.
- FR-ACC-4: Users must be able to delete a trading account.
- FR-ACC-5: Deleting an account must disable copier relations using that account.
- FR-ACC-6: The system must show connection status: connected, disconnected, auth failed, terminal unavailable, broker unavailable, disabled, or locked.

8.3 Copier Relations

- FR-COPY-1: Users must be able to create a master-to-follower copier relation.

- FR-COPY-2: A master account cannot copy to itself.
- FR-COPY-3: Users must be able to enable or disable a copier relation.
- FR-COPY-4: Users must be able to configure lot sizing mode: fixed lot, multiplier, and future equity/risk-based modes.
- FR-COPY-5: The system must copy market entries from master to follower.
- FR-COPY-6: The system must copy SL and TP values where supported.
- FR-COPY-7: The system must copy manual closes.
- FR-COPY-8: The system must detect and handle partial closes.
- FR-COPY-9: The system must detect and handle SL/TP modifications.
- FR-COPY-10: The system must prevent duplicate copying of the same master trade.

8.4 Trade Detection

- FR-DETECT-1: The worker must maintain an internal snapshot of master account positions.
- FR-DETECT-2: The worker must detect new positions, closed positions, SL/TP modification, volume changes, and reversed positions.
- FR-DETECT-3: Each detected event must be normalized into an internal signal format.

```
{
  "event_type": "position_opened",
  "master_account_id": "uuid",
  "master_ticket": "123456",
  "symbol": "XAUUSD",
  "side": "buy",
  "volume": 0.10,
  "open_price": 2350.25,
  "sl": 2345.00,
  "tp": 2360.00,
  "timestamp": "2026-05-20T20:00:00Z"
}
```

8.5 Lot Sizing

- FR-LOT-1: Fixed lot mode must execute the same configured lot size regardless of master trade size.
- FR-LOT-2: Multiplier mode must scale the follower lot based on master lot.
- FR-LOT-3: The system must normalize lot size based on broker constraints: minimum lot, maximum lot, lot step, and margin availability.
- FR-LOT-4: If calculated lot violates broker requirements, the system must round safely, reject the trade, or use configured fallback behavior.

8.6 Symbol Mapping

- FR-SYM-1: The system must support manual symbol mapping.
- FR-SYM-2: The system must support automatic suffix/prefix detection in the future.
- FR-SYM-3: The system must store per-broker symbol metadata including lot step, digits, point size, contract size, spread, and session status.

8.7 Risk Engine

- FR-RISK-1: Max Daily Loss - disable copying, optionally close exposure, log reason, notify user.
- FR-RISK-2: Max Total Loss - allow users to set a maximum total account loss.

- FR-RISK-3: Max Lot Per Trade - cap the maximum copied lot size.
- FR-RISK-4: Max Open Positions - limit total concurrent positions per account.
- FR-RISK-5: Symbol Whitelist - allow only selected symbols.
- FR-RISK-6: Symbol Blacklist - block selected symbols.
- FR-RISK-7: Trade Frequency Limit - limit copied trades per time window.
- FR-RISK-8: Slippage Guard - skip trades if signal age exceeds configured threshold; default 3000ms.
- FR-RISK-9: Equity Stop - stop copying if equity drops below threshold.
- FR-RISK-10: Auto-Flatten - optionally close all exposure when a risk breach occurs.
- FR-RISK-11: News Pause - future feature to pause copying before and after major news.

8.8 Execution Engine

- FR-EXEC-1: Send buy/sell market orders to follower accounts.
- FR-EXEC-2: Include SL/TP during order placement where possible.
- FR-EXEC-3: If broker rejects SL/TP during initial placement, retry SL/TP modification after entry.
- FR-EXEC-4: Capture broker return codes.
- FR-EXEC-5: Classify execution result: success, rejected, partial, skipped by risk, skipped by slippage, failed auth, failed connection, insufficient margin, symbol unavailable, duplicate ignored.
- FR-EXEC-6: Never endlessly retry a failed trade.
- FR-EXEC-7: Retry count must be configurable; MVP default max retries = 1.

8.9 Execution Logs

- FR-LOG-1: Every trade signal must create an execution event.
- FR-LOG-2: Every follower attempt must be logged separately.
- FR-LOG-3: Logs must include master ticket, follower ticket, symbol, side, lots, prices, slippage, latency, status, error message, broker return code, and timestamp.
- FR-LOG-4: Users must be able to filter logs by account, copier relation, symbol, status, and date range.

8.10 Notifications

MVP Notifications

- Email on account disconnected
- Email on risk lockout
- Email on failed execution

Future Notifications

- Telegram
- Discord
- SMS
- Mobile push
- Webhook alerts

8.11 Analytics

MVP Analytics

- Copied trades count
- Successful executions
- Failed executions
- Average latency
- Average slippage
- P&L; per account
- Open positions

Future Analytics

- Win rate
- Profit factor
- Max drawdown
- Equity curve
- Best/worst symbol
- Best/worst time of day
- Risk-adjusted return
- Strategy comparison
- Account leaderboard

8.12 AI Journal

The future AI Journal will summarize daily performance, behavior patterns, overtrading, repeated mistakes, best setups, risk violations, and suggested improvements. It must not give direct financial advice; it should behave as a trade review assistant.

8.13 Billing

Plan	Price	Target User
Starter	\$19/mo	1 master, 2 followers
Pro	\$49/mo	1 master, 10 followers
Scale	\$99/mo	3 masters, 30 followers
Elite	\$149/mo	Higher account limits, dedicated worker
Signal Provider	Custom	Strategy sellers and communities

Recommended pricing should avoid punishing users too heavily per account. A tiered limit model is better than purely linear per-account pricing.

9. Database Schema

9.1 tc_users

```
CREATE TABLE tc_users (  
  id UUID PRIMARY KEY REFERENCES auth.users(id) ON DELETE CASCADE,  
  email VARCHAR(255) UNIQUE NOT NULL,  
  stripe_customer_id VARCHAR(255),  
  subscription_plan VARCHAR(50) DEFAULT 'free',  
  is_active_subscriber BOOLEAN DEFAULT FALSE,  
  account_limit INTEGER DEFAULT 2,  
  follower_limit INTEGER DEFAULT 1,  
  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),  
  updated_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()  
);
```

9.2 trading_accounts

```
CREATE TYPE platform_enum AS ENUM (  
  'mt5', 'mt4', 'ctrader', 'dxtrade', 'matchtrader',  
  'tradelocker', 'ninjatrader', 'tradingview'  
);  
  
CREATE TYPE connection_status_enum AS ENUM (  
  'connected', 'disconnected', 'auth_failed', 'terminal_unavailable',  
  'broker_unavailable', 'disabled', 'locked'  
);  
  
CREATE TYPE account_mode_enum AS ENUM ('hedging', 'netting', 'unknown');  
  
CREATE TABLE trading_accounts (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id UUID NOT NULL REFERENCES tc_users(id) ON DELETE CASCADE,  
  platform platform_enum NOT NULL DEFAULT 'mt5',  
  account_number VARCHAR(64) NOT NULL,  
  broker_server VARCHAR(255) NOT NULL,  
  encrypted_password TEXT NOT NULL,  
  account_label VARCHAR(100),  
  account_mode account_mode_enum DEFAULT 'unknown',  
  connection_status connection_status_enum DEFAULT 'disconnected',  
  balance NUMERIC(14, 2),  
  equity NUMERIC(14, 2),  
  currency VARCHAR(20),  
  leverage INTEGER,  
  last_connected_at TIMESTAMP WITH TIME ZONE,  
  last_error TEXT,  
  is_enabled BOOLEAN DEFAULT TRUE,  
  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()  
);
```

9.3 copier_relations

```
CREATE TYPE risk_mode_enum AS ENUM (
    'multiplier', 'fixed_lot', 'equity_ratio', 'risk_percent'
);

CREATE TABLE copier_relations (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID NOT NULL REFERENCES tc_users(id) ON DELETE CASCADE,
    master_account_id UUID NOT NULL REFERENCES trading_accounts(id) ON DELETE CASCADE,
    follower_account_id UUID NOT NULL REFERENCES trading_accounts(id) ON DELETE CASCADE,
    risk_mode risk_mode_enum DEFAULT 'multiplier',
    multiplier NUMERIC(8, 4) DEFAULT 1.0000,
    fixed_lot_size NUMERIC(10, 2) DEFAULT 0.01,
    copy_sl BOOLEAN DEFAULT TRUE,
    copy_tp BOOLEAN DEFAULT TRUE,
    copy_closes BOOLEAN DEFAULT TRUE,
    copy_modifications BOOLEAN DEFAULT TRUE,
    max_signal_age_ms INTEGER DEFAULT 3000,
    is_enabled BOOLEAN DEFAULT TRUE,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
    CONSTRAINT unique_master_follower_pair UNIQUE (master_account_id, follower_account_id),
    CONSTRAINT check_not_self_copy CHECK (master_account_id <> follower_account_id)
);
```

9.4 symbol_mappings

```
CREATE TABLE symbol_mappings (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID NOT NULL REFERENCES tc_users(id) ON DELETE CASCADE,
    master_account_id UUID REFERENCES trading_accounts(id) ON DELETE CASCADE,
    follower_account_id UUID REFERENCES trading_accounts(id) ON DELETE CASCADE,
    master_symbol VARCHAR(50) NOT NULL,
    follower_symbol VARCHAR(50) NOT NULL,
    is_active BOOLEAN DEFAULT TRUE,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);
```

9.5 risk_profiles

```
CREATE TABLE risk_profiles (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID NOT NULL REFERENCES tc_users(id) ON DELETE CASCADE,
    account_id UUID NOT NULL REFERENCES trading_accounts(id) ON DELETE CASCADE,
    max_daily_loss NUMERIC(14, 2),
    max_total_loss NUMERIC(14, 2),
    min_equity NUMERIC(14, 2),
    max_lot_per_trade NUMERIC(10, 2),
    max_open_positions INTEGER DEFAULT 10,
    max_trades_per_day INTEGER,
    allowed_symbols TEXT[],
    blocked_symbols TEXT[],
    news_pause_enabled BOOLEAN DEFAULT FALSE,
    lock_after_loss BOOLEAN DEFAULT TRUE,
    auto_flatten_enabled BOOLEAN DEFAULT TRUE,
    is_locked BOOLEAN DEFAULT FALSE,
    locked_reason TEXT,
    locked_at TIMESTAMP WITH TIME ZONE,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);
```

9.6 execution_events

```
CREATE TYPE execution_status_enum AS ENUM (
    'pending', 'success', 'failed', 'rejected', 'skipped_risk',
    'skipped_slippage', 'duplicate_ignored', 'partial', 'closed', 'modified'
);

CREATE TABLE execution_events (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID NOT NULL REFERENCES tc_users(id) ON DELETE CASCADE,
    copier_relation_id UUID REFERENCES copier_relations(id) ON DELETE SET NULL,
    master_account_id UUID REFERENCES trading_accounts(id),
    follower_account_id UUID REFERENCES trading_accounts(id),
    event_type VARCHAR(50) NOT NULL,
    master_ticket VARCHAR(100),
    follower_ticket VARCHAR(100),
    symbol_master VARCHAR(50),
    symbol_follower VARCHAR(50),
    side VARCHAR(10),
    requested_lot NUMERIC(10, 2),
    executed_lot NUMERIC(10, 2),
    requested_price NUMERIC(18, 8),
    executed_price NUMERIC(18, 8),
    slippage_points NUMERIC(12, 4),
    latency_ms INTEGER,
    status execution_status_enum NOT NULL,
    broker_return_code VARCHAR(100),
    error_message TEXT,
    raw_payload JSONB,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);
```

9.7 worker_nodes

```
CREATE TABLE worker_nodes (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    worker_name VARCHAR(100) NOT NULL,
    region VARCHAR(100),
    host_identifier VARCHAR(255),
    status VARCHAR(50) DEFAULT 'offline',
    capacity INTEGER DEFAULT 1,
    active_sessions INTEGER DEFAULT 0,
    last_heartbeat_at TIMESTAMP WITH TIME ZONE,
    metadata JSONB,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);
```

9.8 worker_sessions

```
CREATE TABLE worker_sessions (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    worker_node_id UUID REFERENCES worker_nodes(id) ON DELETE SET NULL,
    trading_account_id UUID REFERENCES trading_accounts(id) ON DELETE CASCADE,
    session_status VARCHAR(50) DEFAULT 'starting',
    terminal_path TEXT,
    process_id INTEGER,
    last_heartbeat_at TIMESTAMP WITH TIME ZONE,
    last_error TEXT,
    started_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
    stopped_at TIMESTAMP WITH TIME ZONE
);
```

10. API Design

10.1 Account APIs

POST /api/accounts
GET /api/accounts
GET /api/accounts/:id
PATCH /api/accounts/:id
DELETE /api/accounts/:id
POST /api/accounts/:id/test-connection
POST /api/accounts/:id/start-session
POST /api/accounts/:id/stop-session

10.2 Copier APIs

POST /api/copiers
GET /api/copiers
GET /api/copiers/:id
PATCH /api/copiers/:id
DELETE /api/copiers/:id
POST /api/copiers/:id/enable
POST /api/copiers/:id/disable

10.3 Risk APIs

POST /api/risk-profiles
GET /api/risk-profiles
PATCH /api/risk-profiles/:id
POST /api/risk-profiles/:id/unlock
POST /api/risk-profiles/:id/flatten

10.4 Logs APIs

GET /api/execution-events
GET /api/execution-events/:id

10.5 Worker Internal APIs

POST /internal/workers/register
POST /internal/workers/heartbeat
POST /internal/workers/session-started
POST /internal/workers/session-failed
POST /internal/execution-events

11. Non-Functional Requirements

11.1 Reliability

Metric	Target
Account state polling	250ms-1000ms
Internal routing after signal detection	<=50ms
Follower order dispatch after validation	<=250ms
Initial end-to-end copy target	<=1500ms
Slippage abort threshold	Default 3000ms

11.2 Security

- Broker passwords must be encrypted using AES-256-GCM or equivalent authenticated encryption.
- Encryption keys must not be stored in the database.
- Raw broker passwords must never be logged.
- Frontend must never receive decrypted credentials.
- Supabase RLS must isolate user data.
- Admin users must not be able to view raw credentials.
- All external traffic must use HTTPS/TLS.

11.3 Scalability

- 1 user with 1 master and 5 followers
- 1 user with 5 masters and 50 followers
- 100 users with 500 total accounts
- 1,000+ users after worker orchestration matures

11.4 Observability

- Structured logs
- Worker heartbeat
- Broker error tracking
- Execution latency metrics
- Failed trade alerts
- Account disconnect alerts
- Worker crash alerts

12. Frontend Requirements

12.1 Dashboard Pages

Page Group	Pages
Public Pages	Landing page, pricing page, login, register, forgot password
Authenticated Pages	Dashboard overview, accounts, add account, copiers, create copier relation, risk settings, execution logs, analytics, billing,
Admin Pages	Users, accounts, worker nodes, failed executions, system health, billing overview

12.2 UI Style

- Minimalist
- Professional
- Dark-mode first
- Clean trading dashboard feel

- No clutter
- No generic AI SaaS appearance
- Clear status badges
- Strong tables
- Fast filters
- Sharp account cards
- Trust-building execution logs

13. Worker Engine Requirements

13.1 MT5 Worker MVP

- Initialize MT5
- Login to account
- Fetch account info
- Fetch open positions
- Fetch pending orders
- Fetch symbol metadata
- Place market order
- Modify SL/TP
- Close position
- Shutdown session
- Report errors

13.2 State-Diff Engine

The state-diff engine compares previous_positions to current_positions and classifies the changes as new position, removed position, modified SL, modified TP, volume changed, price changed, or direction changed.

13.3 Signal Normalizer

```
class TradeSignal:
    event_type: str
    master_account_id: str
    master_ticket: str
    symbol: str
    side: str
    volume: float
    open_price: float | None
    sl: float | None
    tp: float | None
    timestamp_ms: int
```

13.4 Execution Pipeline

```
TradeSignal
-> Load copier relations
-> Filter enabled followers
-> Run risk checks
-> Map symbol
-> Calculate lot
-> Validate broker constraints
-> Send order
-> Log result
```

14. Platform Expansion Roadmap

Phase	Focus
Phase 1: MT5 Core	Python MT5 connection, account info, open position reader, demo order placement, demo close, master/follower state diff, b
Phase 2: MT5 SaaS Foundation	Supabase schema, FastAPI API, encrypted credentials, worker sessions, execution logs, basic dashboard
Phase 3: Risk Engine	Max daily loss, max open trades, max lot, symbol whitelist, slippage guard, auto-flatten, lockout
Phase 4: Production Worker Fabric	AWS/cloud deployment, process supervision, worker heartbeats, session restarts, multi-account isolation, worker capacity m
Phase 5: Billing + User Launch	Stripe billing, plans, onboarding, user limits, support tools, private beta
Phase 6: MT4 Support	MQL4 EA, socket/HTTP bridge, local/hosted terminal adapter, Windows worker infrastructure
Phase 7: TradingView Alerts	TradingView webhook to Delta Signal API to risk engine to follower workers
Phase 8: cTrader/DXTrade/MatchTrade/TradeStation	API access and broker support
Phase 9: Developer API	API keys, webhook endpoints, sandbox, signal ingestion, account status, execution logs

15. Technical Build Order

15.1 Immediate Next Step: Python MT5 Script

- Can Python connect to MT5?
- Can it read account info?
- Can it read open positions?
- Can it fetch symbol info?
- Can it place a demo order?
- Can it close a demo order?
- Can it detect a manually placed trade?
- Can it copy that trade to another demo account?

15.2 Sequential Build Plan

- Single account connection script
- Single account position polling script
- Single account order placement script
- Master account listener

- Follower account executor
- Master-to-follower copy script
- Add symbol mapping and lot multiplier
- Add execution logging to local JSON/SQLite
- Move config into Supabase
- Wrap engine with FastAPI
- Build Next.js dashboard

16. Main Technical Risks

16.1 MetaTrader Terminal Dependency

Delta Engine depends on trading terminals, not clean universal broker APIs. This creates challenges around terminal installation, terminal login, broker server selection, terminal crashes, Windows/Wine compatibility, multiple terminals per machine, and account isolation.

16.2 Credential Trust

Cloud-native architecture may require holding encrypted broker credentials. This is powerful but sensitive. Security must be taken seriously from day one.

16.3 Broker Differences

Different brokers have different symbols, suffixes, lot steps, minimum lots, spreads, stop-level rules, execution modes, and margin rules.

16.4 Position Model Differences

MT5 can be hedging or netting. This can affect copying behavior significantly.

16.5 Latency Claims

Delta Engine should not market ultra-low latency early. Early positioning should focus on reliable copying, clear logs, risk protection, zero-install setup, and prop-firm-friendly controls.

17. Definition of MVP Success

- Connect two MT5 demo accounts.
- Mark one account as master and one as follower.
- Detect a manually opened master trade.
- Open a matching follower trade.
- Copy SL/TP.
- Copy manual close.
- Avoid duplicate trades.
- Log errors clearly.
- Block new trades with daily loss lockout.

- Run continuously for at least 24 hours on demo without crashing.

18. Final Product Definition

Delta Engine is a managed trade execution, copying, risk-control, and analytics infrastructure platform for serious multi-account traders.

First: prove MT5 execution.
Second: prove copying.
Third: prove risk controls.
Fourth: prove reliability.
Fifth: build SaaS dashboard.
Sixth: scale into multi-platform infrastructure.

19. Immediate Next Work

The next practical work item is the Python MT5 connection and scripting foundation. The first script should import MetaTrader5, initialize the MT5 terminal, log in to a demo account, print account info, print open positions, print symbol info for EURUSD or XAUUSD, and shut down MT5 cleanly.

```
connect_mt5.py  
place_order.py  
close_order.py  
detect_positions.py  
copy_master_to_follower.py
```

20. Reference Research Notes

This PRD was shaped by a comparative research report on Tradecopia and TradersConnect. The report highlighted the strategic split between local-first, latency-sensitive trade copying and cloud-native, zero-install multi-platform copying. It also highlighted the importance of preemptive risk controls, transparent pricing, platform coverage, and credential-custody trust.